

컴퓨터 비전

동양미래대학교
인공지능소프트웨어학과
백찬은 교수

컨볼루션 신경망을 활용한 컴퓨터 비전

01

LeNet-5 적용하기

02

CNN 적용하기

03

손실함수, 옵티마이저, 데이터 증강

04

전이학습과 백본 모델





LeNet-5 적용하기

(실습) LeNet-5로 MNIST 숫자 인식하기

프로그램 8-1

LeNet-5로 MNIST 인식하기

```
01 import numpy as np
02 import tensorflow as tf
03 import tensorflow.keras.datasets as ds
04
05 from tensorflow.keras.models import Sequential
06 from tensorflow.keras.layers import Conv2D,MaxPooling2D,Flatten,Dropout,Dense
07 from tensorflow.keras.optimizers import Adam
08
09 (x_train,y_train),(x_test,y_test)=ds.mnist.load_data()
10 x_train=x_train.reshape(60000,28,28,1)
11 x_test=x_test.reshape(10000,28,28,1)
12 x_train=x_train.astype(np.float32)/255.0
13 x_test=x_test.astype(np.float32)/255.0
14 y_train=tf.keras.utils.to_categorical(y_train,10)
15 y_test=tf.keras.utils.to_categorical(y_test,10)
16
```

데이터 준비

(실습) LeNet-5로 MNIST 숫자 인식하기

```
17 cnn=Sequential()
18 cnn.add(Conv2D(6,(5,5),padding='same',activation='relu',input_
   shape=(28,28,1)))
19 cnn.add(MaxPooling2D(pool_size=(2,2),strides=2))
20 cnn.add(Conv2D(16,(5,5),padding='valid',activation='relu'))
21 cnn.add(MaxPooling2D(pool_size=(2,2),strides=2))
22 cnn.add(Conv2D(120,(5,5),padding='valid',activation='relu'))
23 cnn.add(Flatten())
24 cnn.add(Dense(units=84,activation='relu'))
25 cnn.add(Dense(units=10,activation='softmax'))
26
27 cnn.compile(loss='categorical_crossentropy',optimizer=Adam
   (learning_rate=0.001),metrics=['accuracy'])
28 cnn.fit(x_train,y_train,batch_size=128,epochs=30,validation_
   data=(x_test,y_test),verbose=2) ①
29
30 res=cnn.evaluate(x_test,y_test,verbose=0)
31 print('정확률=',res[1]*100) ②
```

모델 선택
(신경망 구조
설계)

학습

예측(성능 측정)

(실습) LeNet-5로 MNIST 숫자 인식하기

Epoch 1/30 ①

469/469 - 3s - loss: 0.2555 - accuracy: 0.9212 - val_loss: 0.0794 - val_accuracy: 0.9742 - 3s/epoch - 6ms/step

Epoch 2/30

469/469 - 2s - loss: 0.0671 - accuracy: 0.9786 - val_loss: 0.0536 - val_accuracy: 0.9826 - 2s/epoch - 4ms/step

...

Epoch 29/30

469/469 - 2s - loss: 0.0037 - accuracy: 0.9988 - val_loss: 0.0506 - val_accuracy: 0.9902 - 2s/epoch - 5ms/step

Epoch 30/30

469/469 - 2s - loss: 0.0059 - accuracy: 0.9981 - val_loss: 0.0430 - val_accuracy: 0.9910 - 2s/epoch - 5ms/step

정확률= 99.09999966621399 ②



CNN 적용하기

(실습) CNN으로 CIFAR-10 데이터셋 인식하기

프로그램 8-2

컨볼루션 신경망으로 자연 영상 인식하기

```
01 import numpy as np
02 import tensorflow as tf
03 import tensorflow.keras.datasets as ds
04
05 from tensorflow.keras.models import Sequential
06 from tensorflow.keras.layers import Conv2D,MaxPooling2D,Flatten,Dense,Dropout
07 from tensorflow.keras.optimizers import Adam
08
09 (x_train,y_train),(x_test,y_test)=ds.cifar10.load_data()
10 x_train=x_train.astype(np.float32)/255.0
11 x_test=x_test.astype(np.float32)/255.0
12 y_train=tf.keras.utils.to_categorical(y_train,10)
13 y_test=tf.keras.utils.to_categorical(y_test,10)
```


(실습) CNN으로 CIFAR-10 데이터셋 인식하기

```
14                                     규제 기법인 드롭아웃 적용
15 cnn=Sequential()
16 cnn.add(Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)))
17 cnn.add(Conv2D(32,(3,3),activation='relu'))
18 cnn.add(MaxPooling2D(pool_size=(2,2)))
19 cnn.add(Dropout(0.25))
20 cnn.add(Conv2D(64,(3,3),activation='relu'))
21 cnn.add(Conv2D(64,(3,3),activation='relu'))
22 cnn.add(MaxPooling2D(pool_size=(2,2)))
23 cnn.add(Dropout(0.25))
24 cnn.add(Flatten())
25 cnn.add(Dense(units=512,activation='relu'))
26 cnn.add(Dropout(0.5))
27 cnn.add(Dense(units=10,activation='softmax'))
28
29 cnn.compile(loss='categorical_crossentropy',optimizer=Adam(learning_
    rate=0.001),metrics=['accuracy'])
30 hist=cnn.fit(x_train,y_train,batch_size=128,epochs=100,validation_data=(x_
    test,y_test),verbose=2) ①
31
```

(실습) CNN으로 CIFAR-10 데이터셋 인식하기

```
32 res=cnn.evaluate(x_test,y_test,verbose=0)
33 print('정확률=',res[1]*100) ②
34
35 import matplotlib.pyplot as plt
36
37 plt.plot(hist.history['accuracy']) ③
38 plt.plot(hist.history['val_accuracy'])
39 plt.title('Accuracy graph')
40 plt.ylabel('Accuracy')
41 plt.xlabel('Epoch')
42 plt.legend(['Train','Validation'])
43 plt.grid()
44 plt.show()
45
46 plt.plot(hist.history['loss']) ④
47 plt.plot(hist.history['val_loss'])
48 plt.title('Loss graph')
49 plt.ylabel('Loss')
50 plt.xlabel('Epoch')
51 plt.legend(['Train','Validation'])
52 plt.grid()
53 plt.show()
```

(실습) CNN으로 CIFAR-10 데이터셋 인식하기

Epoch 1/100 ①

391/391 - 3s - loss: 1.6325 - accuracy: 0.3974 - val_loss: 1.2854 - val_accuracy: 0.5411 - 3s/epoch - 9ms/step

Epoch 2/100

391/391 - 3s - loss: 1.2396 - accuracy: 0.5573 - val_loss: 1.0960 - val_accuracy: 0.6054 - 3s/epoch - 7ms/step

...

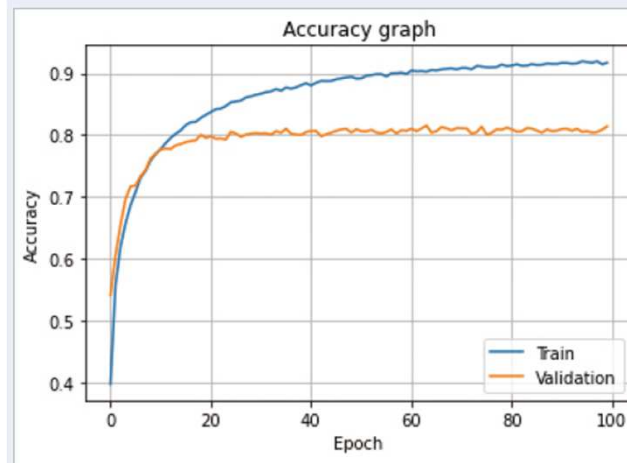
Epoch 100/100

391/391 - 3s - loss: 0.2350 - accuracy: 0.9168 - val_loss: 0.6647 - val_accuracy: 0.8140 - 3s/epoch - 7ms/step

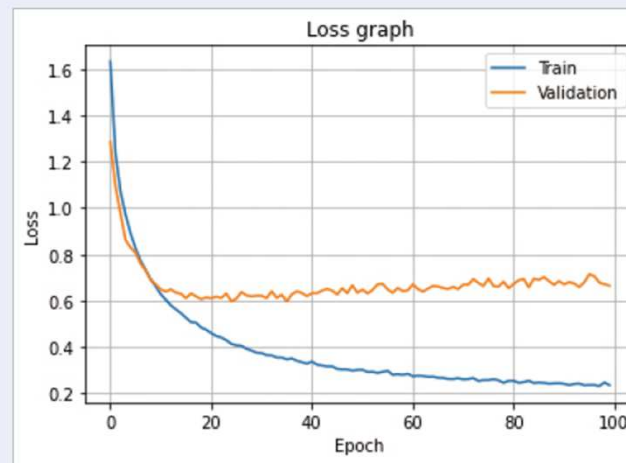
정확률= 81.40000104904175 ②

[프로그램 7-6]의 깊은 다층 퍼셉트론보다 정확률 26.24% 향상

③



④



Tensorflow 프로그래밍

- Tensorflow 프로그래밍의 핵심인 4가지 모듈

- Models API
- Layers API
- Optimizers
- Losses

Models API

- The Model class
- The Sequential class
- Model training APIs
- Model saving & serialization APIs

Layers API

- The base Layer class
- Layer activations
- Layer weight initializers
- Layer weight regularizers
- Layer weight constraints
- Core layers
- Convolution layers
- Pooling layers
- Recurrent layers
- Preprocessing layers
- Normalization layers
- Regularization layers
- Attention layers
- Reshaping layers
- Merging layers
- Locally-connected layers
- Activation layers

Optimizers

- SGD
- RMSprop
- Adam
- Adadelta
- Adagrad
- Adamax
- Nadam
- Ftrl

Losses

- Probabilistic losses
- Regression losses
- Hinge losses for "maximum-margin" classification

그림 8-16 케라스에서 제공하는 models, layers, optimizers, losses 모듈의 API(<https://keras.io/api>)

Tensorflow 프로그래밍 - models 모듈

- 모델 생성하는 models 모듈
 - Sequential과 Functional API가 있음
 - Sequential은 한 갈래 텐서가 끝까지 흐르는 경우에 사용
 - Functional API는 텐서가 여러 갈래로 나뉘는 경우에 사용할 수 있음

```
model=Sequential()  
model.add(Conv2D(32,3,3), input_shape=(32,32,3))  
model.add(Conv2D(64,3,3))  
model.add(Flatten())  
model.add(Dense(10,activation='softmax'))
```

Sequential로 코딩

```
input=Input(shape=(32,32,3))  
x1=Conv2D(32,3,3)(input)  
x2=Conv2D(64,3,3)(x1)  
x3=Flatten()(x2)  
output=Dense(10,activation='softmax')(x3)  
model=Model(input,output)
```

Functional API로 코딩

Tensorflow 프로그래밍 - layers, losses, optimizers 모듈

- 층을 쌓는 layers 모듈
 - 완전연결층 Dense, 컨볼루션층 Conv2D, 최대 풀링층 MaxPooling2D, ...
- 손실 함수를 위한 losses 모듈
 - 평균제곱오차 MSE, 교차 엔트로피 categorical_crossentropy, 분할을 위한 focal, ...
- 옵티마이저를 위한 optimizers 모듈
 - SGD, Adam, AdaGrad, RMSprop 등

(실습) LeNet-5로 만들었던 것을 CNN으로 업그레이드하기

- 프로그램 8-1의 1~15행을 그대로 두고, 이후 네트워크의 모양을 수정하기
- 모델(가중치)을 저장한 후, 숫자 인식 GUI 프로그램을 만들 예정

프로그램 8-3

필기 숫자 인식기 성능 향상하기

01~15행은 [프로그램 8-1]과 같음

```
17 cnn=Sequential()
18 cnn.add(Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)))
19 cnn.add(Conv2D(32,(3,3),activation='relu'))
20 cnn.add(MaxPooling2D(pool_size=(2,2)))
21 cnn.add(Dropout(0.25))
22 cnn.add(Conv2D(64,(3,3),activation='relu'))
23 cnn.add(Conv2D(64,(3,3),activation='relu'))
24 cnn.add(MaxPooling2D(pool_size=(2,2)))
25 cnn.add(Dropout(0.25))
26 cnn.add(Flatten())
27 cnn.add(Dense(units=512,activation='relu'))
28 cnn.add(Dropout(0.5))
29 cnn.add(Dense(units=10,activation='softmax'))
```

(실습) LeNet-5로 만들었던 것을 CNN으로 업그레이드하기

```
30
31 cnn.compile(loss='categorical_crossentropy',optimizer=Adam(learning_
    rate=0.001),metrics=['accuracy'])
32 hist=cnn.fit(x_train,y_train,batch_size=128,epochs=100,validation_data=(x_
    test,y_test),verbose=2)
33
34 cnn.save('cnn_v2.h5')
35
36 res=cnn.evaluate(x_test,y_test,verbose=0)
37 print('정확률=',res[1]*100) ①
```

...

Epoch 100/100

469/469 - 3s - loss: 0.0341 - accuracy: 0.9904 - val_loss: 0.0191 - val_accuracy:
0.9958 - 3s/epoch - 7ms/step

정확률= 99.58000183105469 ①

(실습) PyQt 프로그램 - 지난 강의자료의 프로그램 7-7 일부만 수정

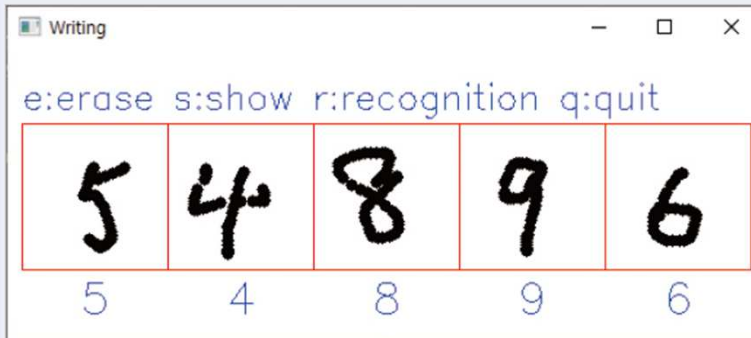
프로그램 8-4

우편번호 인식기 v.2(CNN 버전)

[프로그램 7-7]과 같음 (단 아래와 같이 7행에서 'dmlp_trained.h5'를 'cnn_v2.h5'로 바꾸고, 37행에서 (5,784)을 (5,28,28)로 바꿈)

```
07 model=tf.keras.models.load_model('cnn_v2.h5')
```

```
37 numerals=numerals.reshape(5,28,28,1)
```





손실함수, 옵티마이저, 데이터 증강

손실 함수

- 신경망 초기에는 주로 평균제곱오차(MSE, Mean Squared Error)를 사용

$$J(\mathbf{W}) = \frac{1}{|M|} \sum_{\mathbf{x} \in M} \|\mathbf{y} - \mathbf{o}\|_2^2 \quad (7.23)$$

- 딥러닝에서는 분류를 위해서는 교차 엔트로피(Cross Entropy)를 주로 사용함
- 이외에도 task의 종류에 따라 다양한 손실 함수를 사용함

손실함수

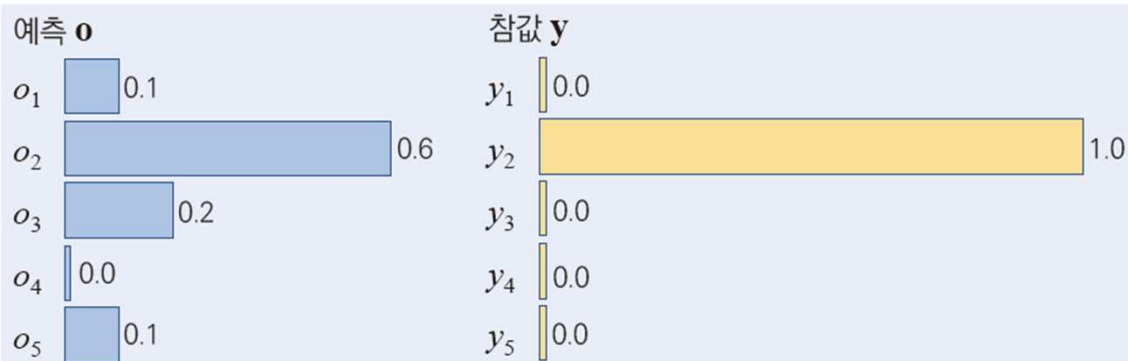
■ 교차 엔트로피

- 참값 벡터 $\mathbf{y} = (y_1, y_2, \dots, y_c)$ 와 예측 벡터 $\mathbf{o} = (o_1, o_2, \dots, o_c)$ 를 확률 분포로 간주
- 교차 엔트로피는 두 확률 분포의 다른 정도를 측정해줌

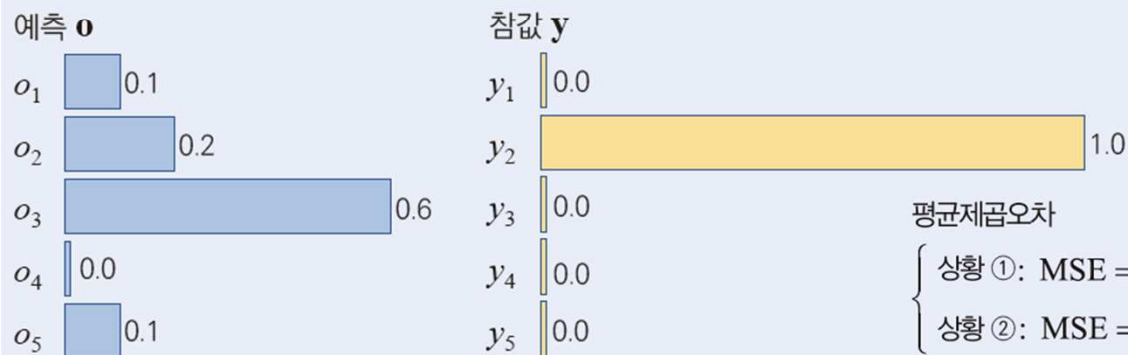
교차 엔트로피:
$$e = -\sum_{i=1,c} y_i \log(o_i) \quad (8.1)$$

손실 함수

■ (예) 평균제곱오차와 교차 엔트로피 손실 함수



(a) 예측이 참값에 근접한 상황①



(b) 예측이 참값에서 벗어난 상황②

그림 8-17 손실 함수 계산

평균제곱오차

$$\begin{cases} \text{상황 ①: } \text{MSE} = (0.0 - 0.1)^2 + (1.0 - 0.6)^2 + (0.0 - 0.2)^2 + (0.0 - 0.0)^2 + (0.0 - 0.1)^2 = 0.22 \\ \text{상황 ②: } \text{MSE} = (0.0 - 0.1)^2 + (1.0 - 0.2)^2 + (0.0 - 0.6)^2 + (0.0 - 0.0)^2 + (0.0 - 0.1)^2 = 1.02 \end{cases}$$

교차 엔트로피

$$\begin{cases} \text{상황 ①: } \text{CE} = -(0.0 \log(0.1) + 1.0 \log(0.6) + 0.0 \log(0.2) + 0.0 \log(0.0) + 0.0 \log(0.1)) = 0.5108 \\ \text{상황 ②: } \text{CE} = -(0.0 \log(0.1) + 1.0 \log(0.2) + 0.0 \log(0.6) + 0.0 \log(0.0) + 0.0 \log(0.1)) = 1.609 \end{cases}$$

손실 함수

■ Focal Loss

- 불균형 데이터(imbalanced data)를 해결하기 위해 Facebook AI 팀이 제안한 손실 함수(2017)
- 문제: 객체탐지 문제에서 대부분의 영역이 배경이며, 일부만 객체임
 - cross entropy loss를 사용하면 학습이 불균형하게 일어남 -> (해결책) 쉽게 맞추는 샘플의 영향은 줄이고, 어려운 샘플에 가중치를 더 주자!(어려운 걸 틀리면 손실이 더 큼) -> Focal Loss 등장

참값이 물체인 화소: $\mathbf{o}=(p, 1-p)$, $\mathbf{y}=(1, 0)$

참값이 배경인 화소: $\mathbf{o}=(p, 1-p)$, $\mathbf{y}=(0, 1)$

$$p_t = \begin{cases} p, & \text{참값이 물체인 화소} \\ 1-p, & \text{참값이 배경인 화소} \end{cases} \quad (8.2)$$

$$e = -\log(p_t) \quad (8.3)$$

$$\text{Focal 손실: } e = -(1-p_t)^\gamma \log(p_t) \quad (8.4)$$

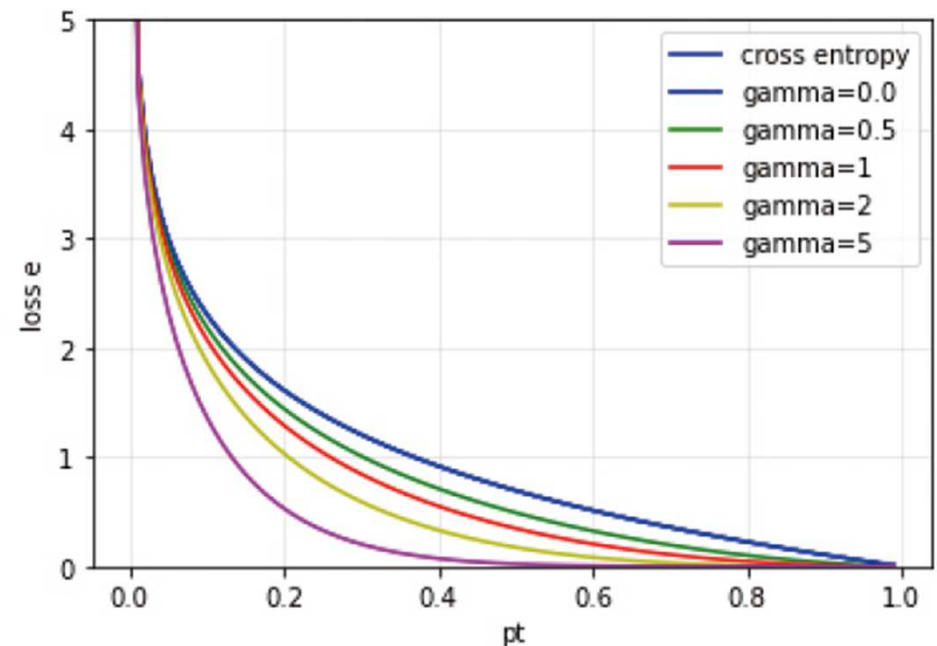


그림 8-18 부류가 2개인 경우의 교차 엔트로피와 Focal 손실 함수

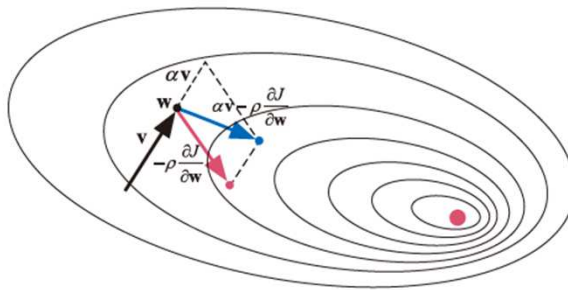
옵티마이저

■ 기본 옵티마이저 SGD를 개선하는 전략으로 모멘텀 적용, 적응적 학습률 적용 등의 방법이 있음

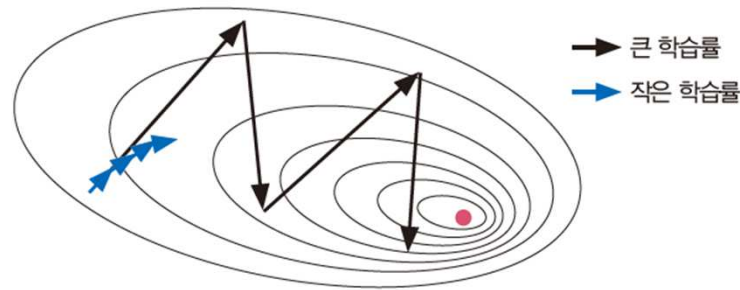
■ 모멘텀

- 이전 운동량이 현재에 영향을 미치는 물리 법칙
- 가중치 변경 이력이 현재 가중치에 영향을 미침

$$\text{SGD: } \mathbf{W} = \mathbf{W} - \rho \frac{\partial J}{\partial \mathbf{W}} \longrightarrow \left. \begin{array}{l} \text{모멘텀을 적용한 SGD: } \mathbf{V} = \alpha \mathbf{V} - \rho \frac{\partial J}{\partial \mathbf{W}} \\ \mathbf{W} = \mathbf{W} + \mathbf{V} \end{array} \right\} \quad (8.5)$$



(a) 모멘텀 아이디어



(b) 학습률에 따른 수렴 특성

그림 8-19 딥러닝에서 사용하는 옵티마이저

```
tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.0, nesterov=False, name="SGD",  
                        **kwargs)
```

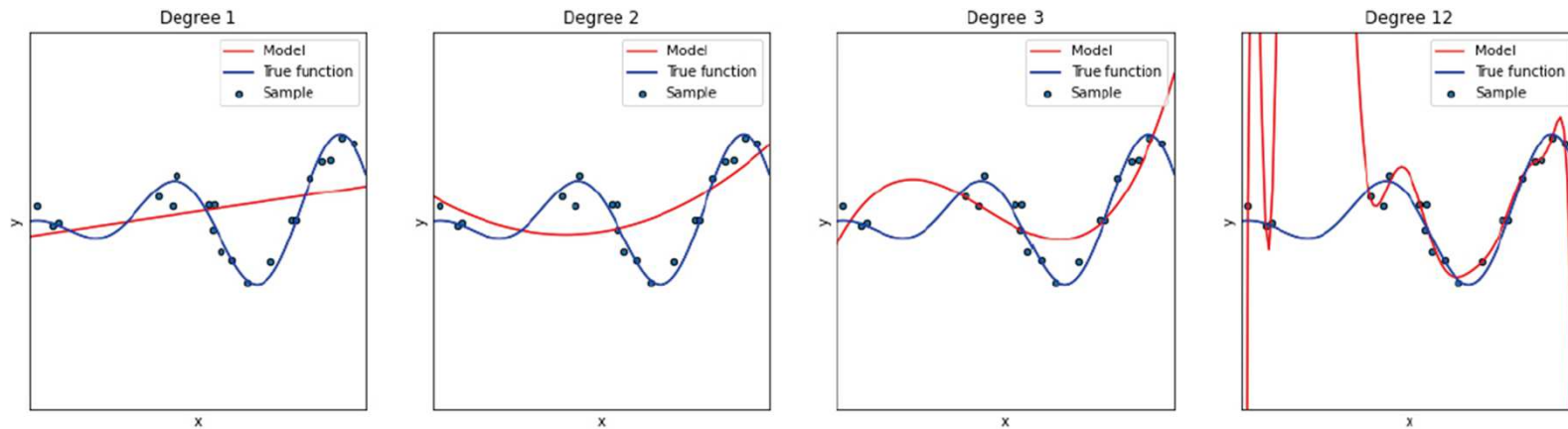
옵티마이저

■ 적응적 학습률

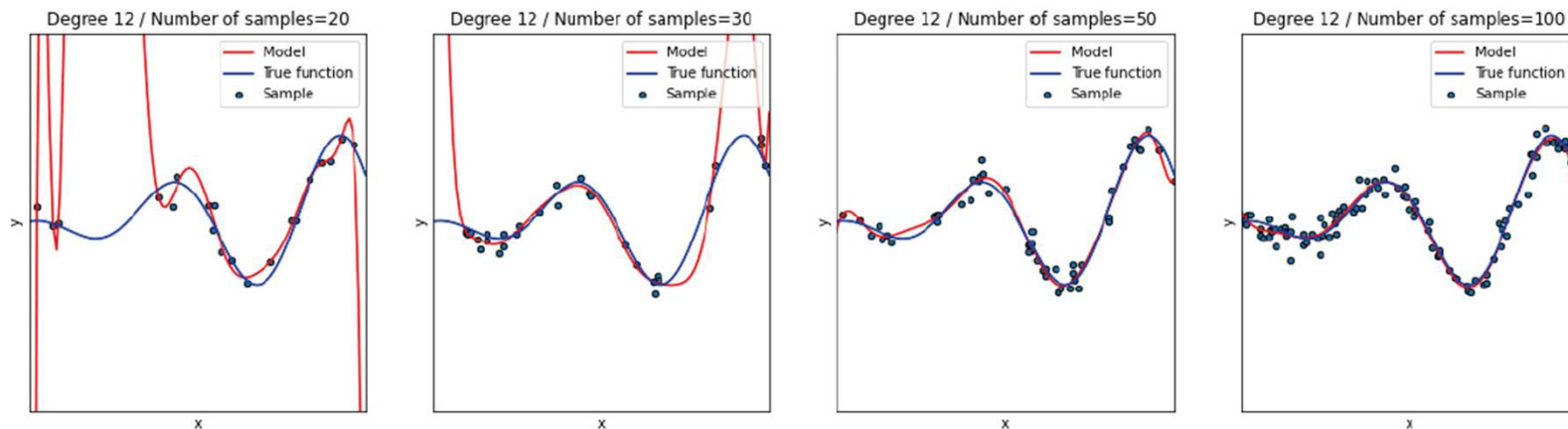
- 식 (8.5)의 표준 SGD는 고정된 학습률 사용
 - 작은 학습률 (0.001, 0.0001과 같은) 사용하여 보수적으로 이동
- 적응적 학습률에서는 상황에 따라 학습률을 조정함[Ruder2016]
 - **AdaGrad**: 이전 그레이디언트를 누적한 정보를 이용해 학습률을 적응적으로 결정
 - **RMSprop**: 그레이디언트를 누적할 때 오래될수록 영향력을 감소하여 AdaGrad 개선
 - **Adam**: RMSprop에 식 (8.5)의 모멘텀을 적용

```
tf.keras.optimizers.Adagrad(learning_rate=0.001, initial_accumulator_value=0.1,  
                             epsilon=1e-07, name="Adagrad", **kwargs)  
tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9, momentum=0.0,  
                             epsilon=1e-07, centered=False, name="RMSprop", **kwargs)  
tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999,  
                          epsilon=1e-07, amsgrad=False, name="Adam", **kwargs)
```


과적합(Overfitting)과 과소적합(Underfitting)



(a) 과소 적합과 과잉 적합 현상



(b) 데이터 증강을 통한 과잉 적합 방지

그림 8-20 과잉 적합 현상과 데이터 증강을 통한 방지

다양한 규제 기법

■ 규제(Regularization)

- 딥러닝과 머신러닝에서 과적합을 방지하기 위해 모델의 복잡도나 자유도를 제한하는 과정
- 일반화 성능을 높임

■ 다양한 규제 기법

▪ 데이터 증강(Data Augmentation)

- 훈련 집합을 조금씩 변형하여 인위적으로 늘림
- 오프라인 방식과 온라인 방식(주로 온라인 사용)

▪ 드롭아웃(Dropout)

- 학습시 임의의 뉴런 일부를 랜덤하게 꺼 일부는 학습이 되지 않도록 배제함
- 학습할 때만 적용하고 예측(추론) 과정에서는 적용 안함

▪ 조기 멈춤(Early Stopping)

- 성능 향상이 없으면 설정한 epoch 이전에 학습을 멈춤

▪

(실습) 영상 증강

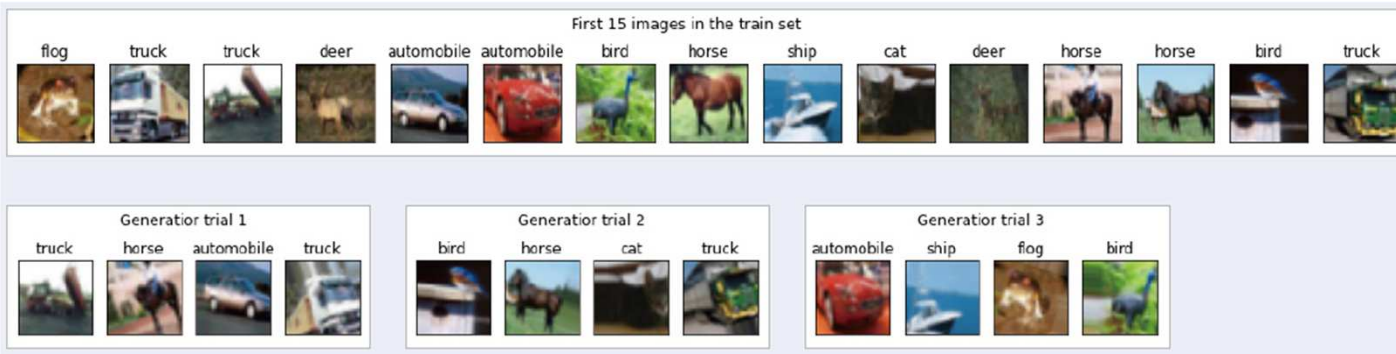
프로그램 8-5

증강된 영상 확인하기

```
01 import tensorflow.keras.datasets as ds
02 from tensorflow.keras.preprocessing.image import ImageDataGenerator
03 import matplotlib.pyplot as plt
04
05 (x_train,y_train),(x_test,y_test)=ds.cifar10.load_data()
06 x_train=x_train.astype('float32'); x_train/=255
07 x_train=x_train[0:15,]; y_train=y_train[0:15,] # 앞 15개에 대해서만 증대 적용
08 class_names=['airplane','automobile','bird','cat','deer','dog','flog','horse',
09              'ship','truck']
10
11 plt.figure(figsize=(20,2))
12 plt.suptitle("First 15 images in the train set")
13 for i in range(15):
14     plt.subplot(1,15,i+1)
15     plt.imshow(x_train[i])
16     plt.xticks([]); plt.yticks([])
17     plt.title(class_names[int(y_train[i])])
18 plt.show()
```

(실습) 영상 증강

```
19 batch_size=4 # 한 번에 생성하는 양(미니 배치)
20 generator=ImageDataGenerator(rotation_range=20.0,width_shift_range=0.2,
    height_shift_range=0.2,horizontal_flip=True)
21 gen=generator.flow(x_train,y_train,batch_size=batch_size)
22
23 for a in range(3):
24     img,label=gen.next() # 미니 배치만큼 생성
25     plt.figure(figsize=(8,2.4))
26     plt.suptitle("Generatior trial «+str(a+1))
27     for i in range(batch_size):
28         plt.subplot(1,batch_size,i+1)
29         plt.imshow(img[i])
30         plt.xticks([]); plt.yticks([])
31         plt.title(class_names[int(label[i])])
32     plt.show()
```





전이학습과 백본 모델

전이 학습(Transfer Learning)과 백본 모델(Backbone Model)

■ 전이 학습(Transfer Learning)

- 어떤 도메인의 데이터로 학습한 모델을 다른 도메인에 적용하여 성능을 향상하는 방법

■ 백본 모델(Backbone Model)

- CNN, Vision Transformer와 같은 특징 추출기(Feature Extractor) 역할을 하는 기본 신경망 구조
- 즉, 이미지 데이터를 다루는 모델에서는 전체 모델의 뼈대로서 이미지를 입력받아 의미 있는 특징(feature map)을 뽑아내는 부분을 뜻함

분야	백본
객체 탐지 (Object Detection)	ResNet, CSPDarkNet
분할 (Segmentation)	ResNet, Xception, Swin
영상 분류 (Video Classification)	3D ResNet
멀티모달 (Vision+Language)	CLIP-ViT

백본 모델

■ 텐서플로가 제공하는 사전 학습 모델

- 전이 학습할 때는 보통 이들 모델을 백본으로 사용

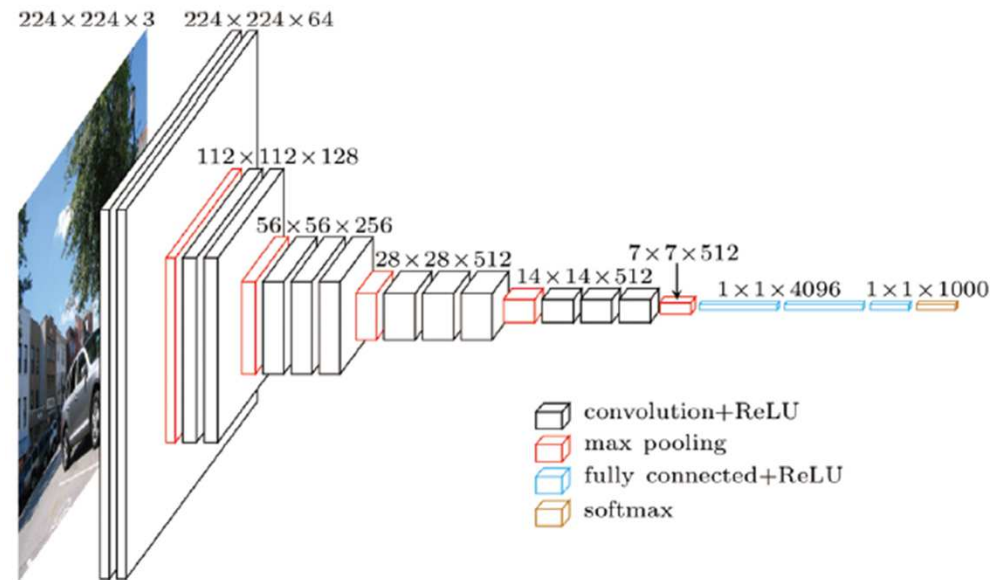
Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7

그림 8-22 텐서플로에서 제공하는 사전 학습 모델의 일부(<https://keras.io/api/applications>)

백본 모델

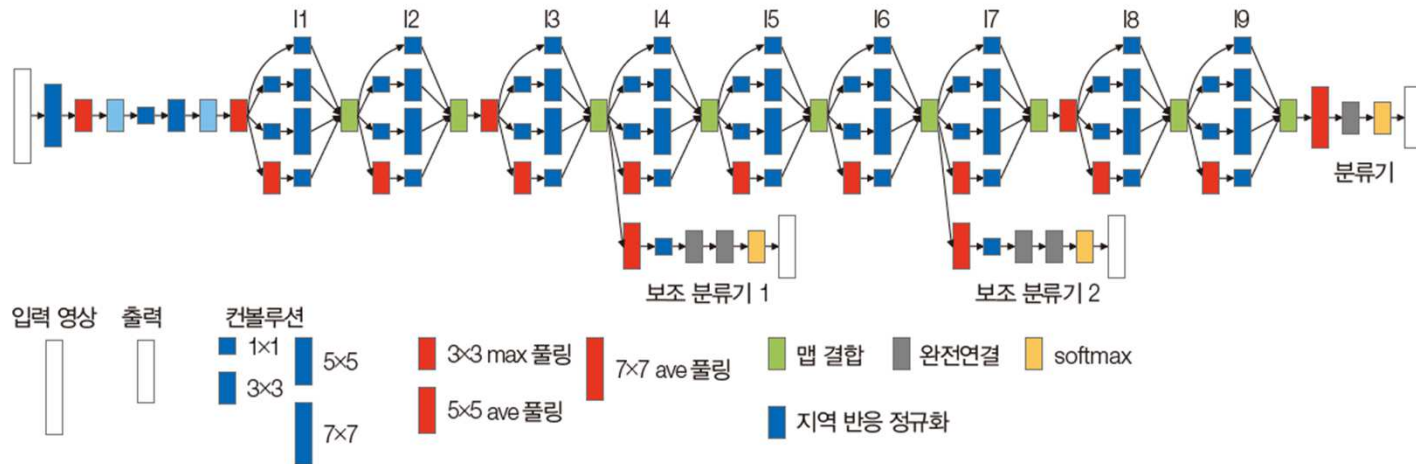
■ 유명한 사전 학습 모델

- VGGNet: 옥스퍼드 대학이 제작. 3*3 작은 마스크 사용하고, 학습할 수 있는 층의 깊이를 16으로 깊게 만듦
- GoogLeNet: 구글이 제작. 네트워크 속의 네트워크 아이디어 적용 - Inception 모듈
- ResNet: 마이크로소프트가 제작. 지름길(숏컷) 연결 적용

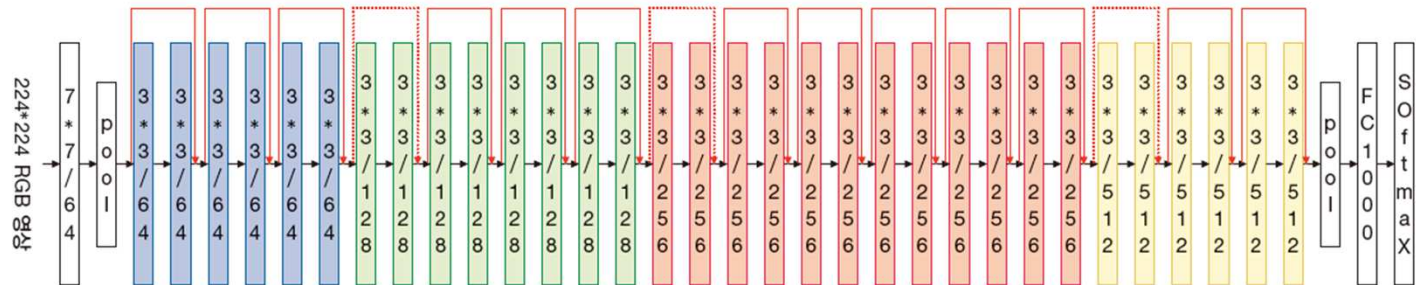


(a) VGGNet

백본 모델



(b) GoogLeNet



(c) ResNet

그림 8-23 VGGNet, GoogLeNet, ResNet의 구조

(실습) 사전 학습 모델(ResNet50)로 영상 인식하기

프로그램 8-6

ResNet50으로 자연 영상 인식하기

```
01 import cv2 as cv
02 import numpy as np
03 from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_
   input, decode_predictions
04
05 model=ResNet50(weights='imagenet')
06
07 img=cv.imread('rabbit.jpg')
08 x=np.reshape(cv.resize(img,(224,224)),(1,224,224,3))
09 x=preprocess_input(x)
10
11 preds=model.predict(x)
12 top5=decode_predictions(preds,top=5)[0]
13 print('예측 결과:',top5)
14
15 for i in range(5):
16     cv.putText(img,top5[i][1]+' ':''+str(top5[i][2]),(10,20+i*20),cv.FONT_HERSHEY_
       SIMPLEX,0.5,(255,255,255),1)
17
18 cv.imshow('Recognition result',img)
19
20 cv.waitKey()
21 cv.destroyAllWindows()
```

(실습) 사전 학습 모델(ResNet50)로 영상 인식하기

예측 결과: [('n02325366', 'wood_rabbit', 0.74258107), ('n02326432', 'hare', 0.24038698), ('n02328150', 'Angora', 0.008831766), ('n01877812', 'wallaby', 0.0026911795), ('n02356798', 'fox_squirrel', 0.0012295933)]



(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기

■ Stanford dogs 데이터셋

- 미세 분류 fine-grained classification 문제
- 부류(개의 품종)가 120개이고 영상이 20,580장

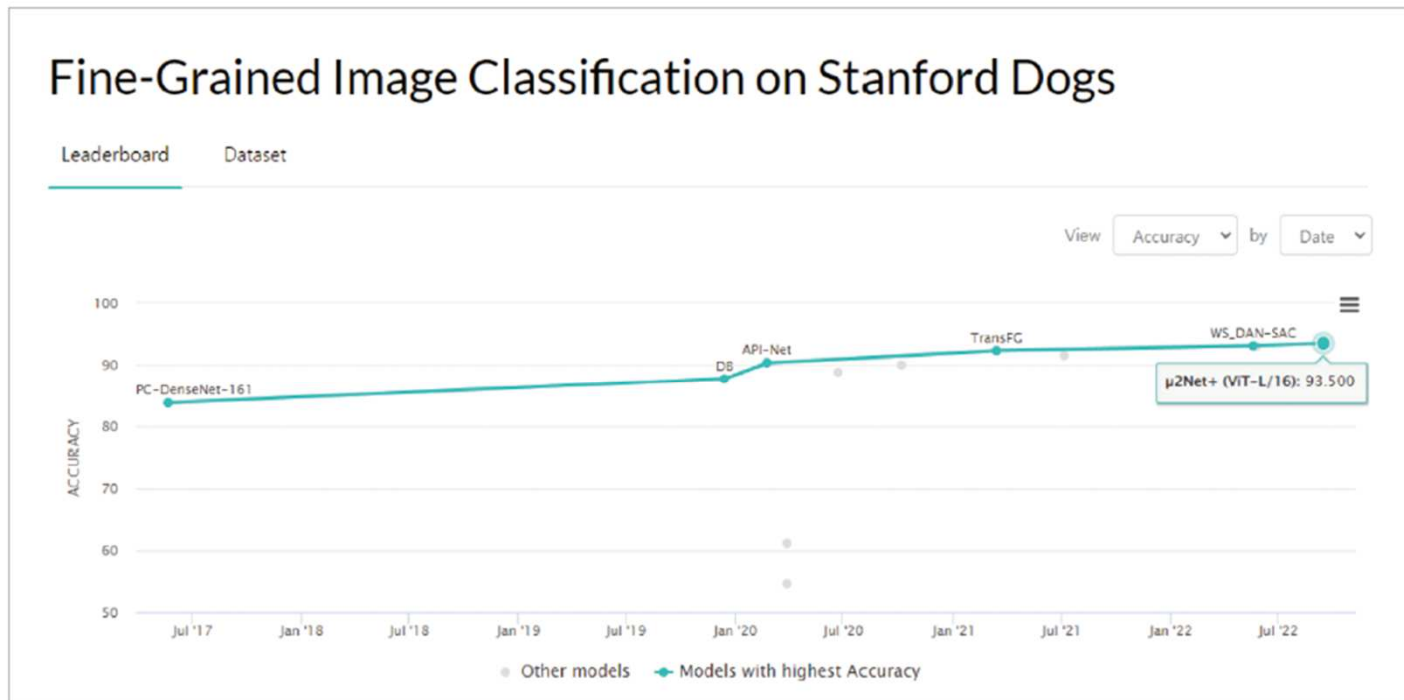


그림 8-24 Stanford dogs 데이터셋의 SOTA 성능

(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기

■ Stanford dogs 데이터셋 설치

1. 소스코드가 있는 폴더에 datasets/Stanford_dogs 폴더를 만든다.
2. <http://vision.stanford.edu/aditya86/ImageNetDogs>에 접속해 images.tar, annotations.tar, list.tar 파일을 1에서 만든 Stanford_dogs 폴더에 다운로드한다.
3. 세 파일의 압축을 해제하고 annotations, images, list 폴더를 확인한다.
4. images/images 폴더를 열어 'n02085620-Chihuahua'를 비롯한 120개 폴더가 있는지 확인한다. 폴더 이름 끝에 있는 Chihuahua는 개의 품종, 즉 부류 이름이다.



그림 8-25 Stanford dogs 데이터셋(왼쪽부터 Chihuahua, Japanese_spaniel, Maltese_dog, Pekinese, Shih-Tzu, Blenheim_spaniel)

(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기

프로그램 8-7

DenseNet121로 견종 인식하기

```
01 from tensorflow.keras.models import Sequential
02 from tensorflow.keras.layers import Flatten,Dense,Dropout,Rescaling
03 from tensorflow.keras.optimizers import Adam
04 from tensorflow.keras.applications.densenet import DenseNet121
05 from tensorflow.keras.utils import image_dataset_from_directory
06 import pathlib
07
08 data_path=pathlib.Path('datasets/stanford_dogs/images/images')
09
10 train_ds=image_dataset_from_directory(data_path,validation_split=0.2,subset
   ='training',seed=123,image_size=(224,224),batch_size=16) ㉠
11 test_ds=image_dataset_from_directory(data_path,validation_split=0.2,subset
   ='validation',seed=123,image_size=(224,224),batch_size=16)
12
13 base_model=DenseNet121(weights='imagenet',include_top=False,input_shape
   =(224,224,3))
14 cnn=Sequential()
15 cnn.add(Rescaling(1.0/255.0))
16 cnn.add(base_model)
17 cnn.add(Flatten())
18 cnn.add(Dense(1024,activation='relu'))
19 cnn.add(Dropout(0.75))
20 cnn.add(Dense(units=120,activation='softmax'))
```


(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기

```
21
22 cnn.compile(loss='sparse_categorical_crossentropy',optimizer=Adam(learning_
    rate=0.000001),metrics=['accuracy'])
23 hist=cnn.fit(train_ds,epochs=200,validation_data=test_ds,verbose=2) ②
24
25 print('정확률=',cnn.evaluate(test_ds,verbose=0)[1]*100) ③
26
27 cnn.save('cnn_for_stanford_dogs.h5')          # 미세 조정된 모델을 파일에 저장
28
```

(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기

```
29 import pickle
30 f=open('dog_species_names.txt','wb')
31 pickle.dump(train_ds.class_names,f)
32 f.close()
33
34 import matplotlib.pyplot as plt
35
36 plt.plot(hist.history['accuracy']) ④
37 plt.plot(hist.history['val_accuracy'])
38 plt.title('Accuracy graph')
39 plt.ylabel('Accuracy')
40 plt.xlabel('Epoch')
41 plt.legend(['Train','Validation'])
42 plt.grid()
43 plt.show()
44
45 plt.plot(hist.history['loss']) ⑤
46 plt.plot(hist.history['val_loss'])
47 plt.title('Loss graph')
48 plt.ylabel('Loss')
49 plt.xlabel('Epoch')
50 plt.legend(['Train','Validation'])
51 plt.grid()
52 plt.show()
```


(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기

Found 20580 files belonging to 120 classes. ①

Using 16464 files for training.

Found 20580 files belonging to 120 classes.

Using 4116 files for validation.

Epoch 1/200 ②

1029/1029 - 181s - loss: 6.5628 - accuracy: 0.0111 - val_loss: 4.6525 - val_accuracy: 0.0355 - 181s/epoch - 176ms/step

Epoch 2/200

1029/1029 - 172s - loss: 4.9830 - accuracy: 0.0214 - val_loss: 4.4681 - val_accuracy: 0.0717 - 172s/epoch - 168ms/step

Epoch 3/200

1029/1029 - 170s - loss: 4.6948 - accuracy: 0.0315 - val_loss: 4.3172 - val_accuracy: 0.1273 - 170s/epoch - 166ms/step

...

Epoch 199/200

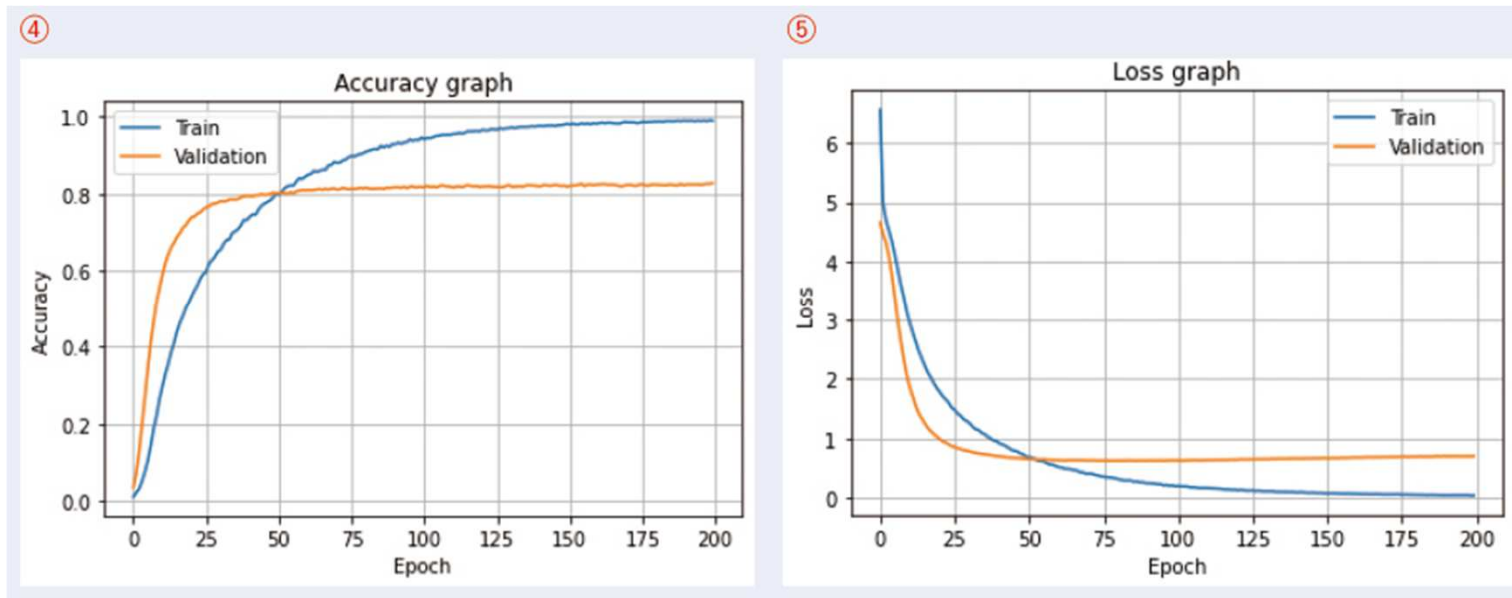
1029/1029 - 167s - loss: 0.0360 - accuracy: 0.9907 - val_loss: 0.6933 - val_accuracy: 0.8258 - 167s/epoch - 163ms/step

Epoch 200/200

1029/1029 - 168s - loss: 0.0378 - accuracy: 0.9897 - val_loss: 0.6995 - val_accuracy: 0.8273 - 168s/epoch - 163ms/step

정확률= 82.72594809532166 ③

(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기



(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기 - GUI 구현

프로그램 8-8 견종 인식 프로그램 구현하기

```
01 import cv2 as cv
02 import numpy as np
03 import tensorflow as tf
04 import winsound
05 import pickle
06 import sys
07 from PyQt5.QtWidgets import *
08
09 cnn=tf.keras.models.load_model('cnn_for_stanford_dogs.h5') # 모델 읽기
10 dog_species=pickle.load(open('dog_species_names.txt','rb')) # 견종 이름
11
12 class DogSpeciesRecognition(QMainWindow):
13     def __init__(self) :
14         super().__init__()
15         self.setWindowTitle('견종 인식')
16         self.setGeometry(200,200,700,100)
17
18         fileButton=QPushButton('강아지 사진 열기',self)
19         recognitionButton=QPushButton('품종 인식',self)
20         quitButton=QPushButton('나가기',self)
21
22         fileButton.setGeometry(10,10,100,30)
23         recognitionButton.setGeometry(110,10,100,30)
24         quitButton.setGeometry(510,10,100,30)
25
26         fileButton.clicked.connect(self.pictureOpenFunction)
27         recognitionButton.clicked.connect(self.recognitionFunction)
28         quitButton.clicked.connect(self.quitFunction)
```

(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기 - GUI 구현

```
29
30 def pictureOpenFunction(self):
31     fname=QFileDialog.getOpenFileName(self,'강아지 사진 읽기','./')
32     self.img=cv.imread(fname[0])
33     if self.img is None: sys.exit('파일을 찾을 수 없습니다.')
34
35     cv.imshow('Dog image',self.img)
36
37 def recognitionFunction(self):
38     x=np.reshape(cv.resize(self.img,(224,224)),(1,224,224,3))
39     res=cnn.predict(x)[0] # 예측
40     top5=np.argsort(-res)[:5]
41     top5_dog_species_names=[dog_species[i] for i in top5]
42     for i in range(5):
43         prob='('+str(res[top5[i]])+')'
44         name=str(top5_dog_species_names[i]).split('-')[1]
45         cv.putText(self.img,prob+name,(10,100+i*30),cv.FONT_HERSHEY_
46             SIMPLEX,0.7,(255,255,255),2)
47     cv.imshow('Dog image',self.img)
48     winsound.Beep(1000,500)
49
50 def quitFunction(self):
51     cv.destroyAllWindows()
52     self.close()
53
54 app=QApplication(sys.argv)
55 win=DogSpeciesRecognition()
56 win.show()
57 app.exec_()
```

(실습) 사전 학습 모델(DenseNet121)로 견종 분류하기 - GUI 구현

